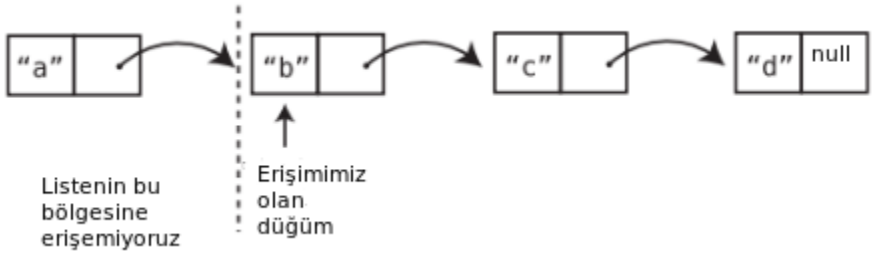
Algoritmalar ve Veri Yapıları Bootcamp Ödev 6.

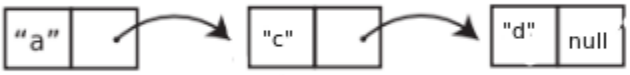
1. Diyelim ki klasik tek bağlantılı bir linkli listenin (singly linked list) ortasındaki (ne başındaki ne de sonundaki) bir düğüme (Node) erişiminiz var, ancak bağlantılı listenin kendisine, yani head'ine, erişiminiz yok. Bir başka deyişle, bir düğüme işaret eden bir göstergeçiniz (pointer) var, ancak LinkedList'in kendisine (head'e) erişimimizi sağlayacak bir göstergeçimiz yok. Dolayısıyla, listenin en başından başlayıp hoplayarak ilerleme şansımız da yok, ancak elimizde göstergeçimizin olduğu düğümden başlayarak listenin sonuna doğru hoplayarak ilerleyebiliyoruz. Bu durumda, bu düğümün bağlantısını izlerseniz, bu orta düğümden başlayarak listenin sonuna kadar tüm öğeleri bulabilirsiniz, ancak listede bu düğümden önce gelen düğümleri bulmanın bir yolu yok.

Bu düğümü listeden silecek olan ve O(1) zaman karmaşıklığında çalışan bir Algoritmayı yazınız. Algoritmanız işini bitirdiğinde geriye, bu düğüm kaldırılmış olarak, kalan listenin tamamı eksiksiz kalmalıdır. Aşağıda algoritmanız öncesindeki durum ve sonrasında beklenen durum birer şekil ile gösterilmiştir.

Mevcut Durum:



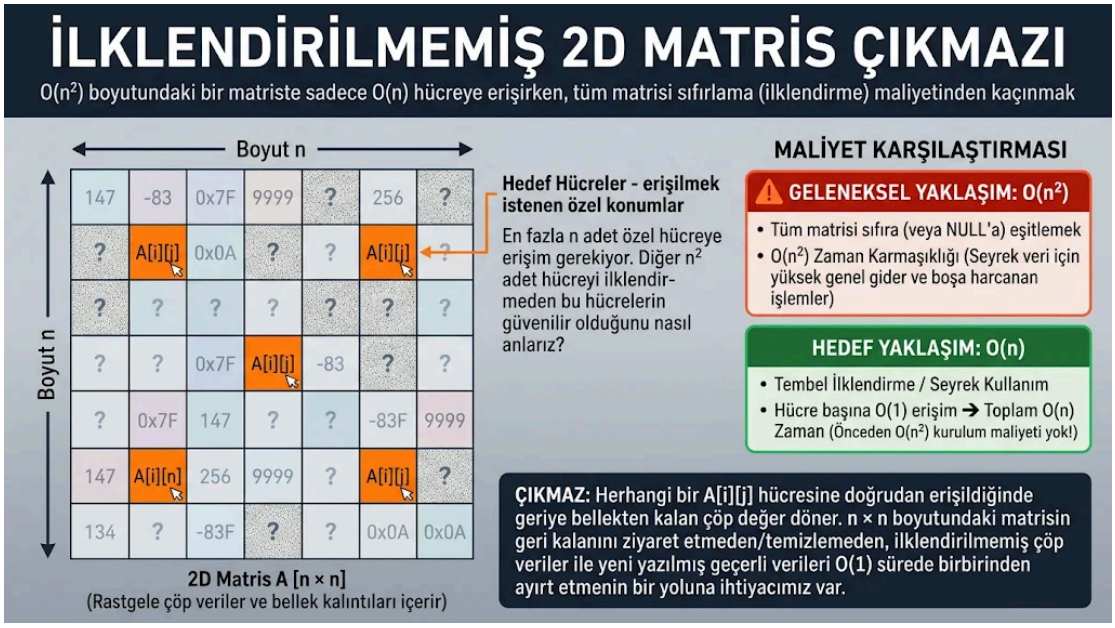
İstenen Durum:



2. Diyelim ki, tamsayıları depolamak için kullanmak istediğimiz iki boyutlu (n×n) bir A dizimiz var, ancak tüm hücrelerine ilk değerlerini atamak için $O(n^2)$ zaman karmaşıklığını harcamak istemiyoruz, zira algoritmamızda bu hücrelerden en fazla n tanesini kullanacağımızı biliyoruz ve algoritmamızın kendisi zaten O(n) zamanında çalışıyor (A'yı ilk değerleme süresini saymazsak). O(n) zamanda çalışan bir algoritmanın koşmaya başlamasından önce kullanacağı veri yapısının ilk değerlenmesi için $O(n^2)$ zaman harcamak bize anlamsız geliyor.

İstiyoruz ki, iki boyutlu A dizisinin elemanlarının ilk değerlerini vermekle hiç uğraşmayalım, bu iki boyutlu dizi tamamen anlamsız ve rastgele değerlerle başlayabilsin. Sonrasında, A'nın hücreleri gerçek değerler aldıkça (A'nın en fazla n adet hücresinin gerçek değer alabileceğini önceden biliyoruz, diğer n^2-n hücre hiç bir zaman gerçek değer almayacak ve hep anlamsız/rastgele değerler içerecek).

Aşağıdaki şekil karşı karşıya olduğumuz durum hakkında bir fikir verecektir.



Bu şartlar altında, tek boyutlu ve uzay karmaşıklığı O(n) olan bir veri yapısı (normal/statik dizi/array, tek bağıntılı linkli liste/singly linked list, ya da yığıt/stack) A'nın işe yarar hücre değerlerini bu veri yapısında tutmak, ama A'da bulundukları yer bilgisini de (iki boyutlu dizide hangi satır ve sütunda bulundukları bilgisi) kaybetmemek istiyoruz. O(n) zaman karmaşıklığında koşacak olan algoritmamız A'nın kendisine lazım olan hücre değerlerinin A'da hangi koordinatlarda -(i, j)- oturduğunu da bir sebepten ötürü bilebilmeli) Seçtiğiniz tek boyutlu veri yapısının her bir gözünde (i, j, k) tam sayı üçlüleri tutarak/saklayarak amacımıza nasıl ulaşılabileceğini anlatın ve koşacak olan algoritmanın ihtiyaç duyacağı operasyonları tanımlayarak algoritmalarını pseudocode olarak yazınız.

Bu problem için en uygun olan tek boyutlu veri yapısının hangisi (normal/statik dizi/array, tek bağıntılı linkli liste/singly linked list, ya da yığıt/stack) olduğunu düşünüyorsunuz? Neden? Neden diğerleri değil? Açıklayınız.